

1 RL Fundamentals

Definition: State $s \in S$, Action $a \in A$, Reward $r \in \mathbb{R}$, Policy $\pi(a|s)$

Value Functions: $v_\pi(s) = \mathbb{E}_\pi[G_t|s_t = s]$ $q_\pi(s, a) = \mathbb{E}_\pi[G_t|s_t = s, a_t = a]$

Returns: $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ (discounted), $G_t = \sum_{k=0}^{\infty} r_{t+k+1}$ (undiscounted)

Connection: $v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a)$ (stochastic), $v_\pi(s) = q_\pi(s, \pi(s))$ (deterministic)

Bellman Equations: $v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma v_\pi(s')]$ $q_\pi(s, a) = \sum_{s', r} p(s', r|s, a)[r + \gamma \sum_{a'} \pi(a'|s') q_\pi(s', a')]$

Optimality: π^* optimal if $v_{\pi^*}(s) \geq v_\pi(s), \forall s, \pi$
 $v^*(s) = \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v^*(s')]$ $q^*(s, a) = \sum_{s', r} p(s', r|s, a)[r + \gamma \max_{a'} q^*(s', a')]$

2 MDP & Markov Property

Markov Property: $p(s', r|s_t, a_t) = p(s', r|s_t, a_t, \dots, s_0, a_0)$

Transition Prob: $P: S \times \mathcal{R} \times S \times A \rightarrow [0, 1]$

POMDP: Partially Observable MDP with observations $\omega \in \Omega$, $p(s|\omega)$

3 Multi-Armed Bandits

Objective: Minimize total regret $\mathbb{E}[\sum_{t=1}^T (q(a^*) - q(a_t))]$

Action-Value Est: $q_t(a) = \frac{1}{N_t(a)} \sum_{t=1}^T r_t \mathbf{1}(a_t = a)$

Update Rule: $q_{n+1} = q_n + \alpha(r_n - q_n)$ where $\alpha = 1/n$ or constant

3.1 ϵ -Greedy

With prob $1 - \epsilon$: $a_t = \arg \max_a q_t(a)$ With prob ϵ : random action
Decay: $\epsilon_{t+1} = (1 - t/T)\epsilon_{\text{init}}$

3.2 Softmax

$$p(a_t = a) = \frac{\exp(q_t(a)/\tau)}{\sum_{a'} \exp(q_t(a')/\tau)}$$

3.3 UCB

$$a_t = \arg \max_a \left[q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

4 Dynamic Programming

Requires full MDP model. Complexity: $O(|S|^2|A|)$ per iteration

4.1 Policy Iteration

1. **Policy Evaluation:** Solve $v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a)[r + \gamma v_\pi(s')]$ 2. **Policy Improvement:** $\pi'(s) = \arg \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v_\pi(s')]$ 3. Repeat until convergence

4.2 Value Iteration

$v_{k+1}(s) = \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v_k(s')]$ Extract policy:
 $\pi(s) = \arg \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v(s')]$

5 Monte Carlo Methods

Model-free, uses complete episodes. Low bias, high variance.

Update: $v_\pi(s_t) = v_\pi(s_t) + \alpha(G_t - v_\pi(s_t))$

First-visit vs Every-visit: Average returns from first/all visits to s

5.1 On-Policy Control

Use ϵ -greedy on $q_\pi(s, a)$. Policy improvement guaranteed for ϵ -greedy.

5.2 Off-Policy Control

Importance Sampling: $\rho_{t:T-1} = \prod_{\tau=t}^{T-1} \frac{\pi(a_\tau|s_\tau)}{b(a_\tau|s_\tau)} v_\pi(s) = \mathbb{E}_b[\rho_{t:T-1} G_t | s_t = s]$

6 Temporal Difference Learning

Model-free, updates every step. Higher bias, lower variance.

TD Error: $\delta_t = r_{t+1} + \gamma v_\pi(s_{t+1}) - v_\pi(s_t)$

TD Target: $r_{t+1} + \gamma v_\pi(s_{t+1})$

6.1 TD(0) Prediction

$$v_\pi(s_t) = v_\pi(s_t) + \alpha \delta_t$$

6.2 SARSA (On-Policy)

$$q_\pi(s_t, a_t) = q_\pi(s_t, a_t) + \alpha[r_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1}) - q_\pi(s_t, a_t)]$$

6.3 Q-Learning (Off-Policy)

$$q_\pi(s_t, a_t) = q_\pi(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_a q_\pi(s_{t+1}, a) - q_\pi(s_t, a_t)]$$

6.4 Expected SARSA

$$q_\pi(s_t, a_t) = q_\pi(s_t, a_t) + \alpha[r_{t+1} + \gamma \mathbb{E}_\pi[q_\pi(s_{t+1}, a)] - q_\pi(s_t, a_t)]$$

6.5 Double Q-Learning

Maintain q_1, q_2 . Update one with prob 0.5: $q_1(s_t, a_t) = q_1(s_t, a_t) + \alpha[r_{t+1} + \gamma q_2(s_{t+1}, \arg \max_a q_1(s_{t+1}, a)) - q_1(s_t, a_t)]$

7 Model-Based RL

Learn model $p(s', r|s, a)$, then plan.

Dyna-Q: Combine learning + planning 1. Real experience: Update $q(s, a)$ 2. Model learning: Store $(s, a) \rightarrow (s', r)$ 3. Planning: Sample from model, update $q(s, a)$

Prioritized Sweeping: Priority = $|r + \gamma \max_{a'} q(s', a') - q(s, a)|$

7.1 MCTS

1. **Selection:** Tree policy to leaf 2. **Expansion:** Add child nodes 3. **Simulation:** Rollout policy 4. **Backup:** MC con-

trol update

8 Function Approximation

For large state/action spaces. Parameterize $\hat{v}(s, w)$ or $\hat{q}(s, a, w)$.

MSE Objective: $J(w) = \mathbb{E}_\pi[(v_\pi(s) - \hat{v}(s, w))^2]$

Gradient: $w_{t+1} = w_t + \alpha[v_\pi(s_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$

8.1 MC with Function Approx

$$w_{t+1} = w_t + \alpha[G_t - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$$

8.2 Semi-Gradient TD(0)

$$w_{t+1} = w_t + \alpha[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)] \nabla_w \hat{v}(s_t, w_t)$$

8.3 Linear Methods

$\hat{v}(s, w) = w^T x(s)$, $\nabla_w \hat{v}(s, w) = x(s)$ Guaranteed convergence for MC and semi-gradient TD(0).

Convergence Condition: Matrix $A = \mathbb{E}[x(s_t)(x(s_t) - \gamma x(s_{t+1}))^T]$ must be positive definite.

8.4 Deadly Triad

Instability when combining: Function Approximation + Bootstrapping + Off-Policy

9 Deep Q-Networks (DQN)

Experience Replay: Store (s, a, r, s') , sample mini-batch

Target Network: $\hat{q}(s', a', w^-)$ updated every C steps
 $L(w) = \mathbb{E}[(r + \gamma \max_{a'} \hat{q}(s', a', w^-) - \hat{q}(s, a, w))^2]$

10 n-Step Methods

n-Step Return: $G_{t:t+n} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n v(s_{t+n})$

n-Step TD: $v(s_t) = v(s_t) + \alpha[G_{t:t+n} - v(s_t)]$

10.1 TD(λ)

λ -Return: $G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$

Eligibility Traces: $z_t = \gamma \lambda z_{t-1} + \nabla_w \hat{v}(s_t, w)$ $w_{t+1} = w_t + \alpha \delta_t z_t$

11 Policy Gradient

Parameterize policy $\pi(a|s, \theta)$ directly.

Objective: $J(\theta) = \sum_s \rho(s) v_\pi(s)$ where $\rho(s)$ is stationary distribution

Policy Gradient Theorem: $\nabla_\theta J(\theta) = \mathbb{E}_\pi[q_\pi(s, a) \nabla_\theta \ln \pi(a|s, \theta)]$

11.1 REINFORCE

$$\theta_{t+1} = \theta_t + \alpha \gamma^t G_t \nabla_\theta \ln \pi(a_t|s_t, \theta_t)$$

With Baseline: Replace G_t with $G_t - b(s_t)$ Common choice: $b(s_t) = \hat{v}(s_t, w)$

11.2 Actor-Critic

Actor: Policy $\pi(a|s, \theta)$ **Critic:** Value function $\hat{v}(s, w)$
 $\theta_{t+1} = \theta_t + \alpha_{\theta} \delta_t \nabla_{\theta} \ln \pi(a_t|s_t, \theta_t)$ $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w \hat{v}(s_t, w_t)$

11.3 PPO (Proximal Policy Optimization)

Clipped Objective: $L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$ where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

Advantage: $\hat{A}_t = \hat{Q}(s_t, a_t) - \hat{V}(s_t)$

12 Advanced Methods

12.1 Distributional RL

Model value distribution instead of expectation. Can handle multi-modalities.

12.2 DDPG

Actor-critic for continuous actions. Deterministic policy gradient. **Actor:** $a = \mu(s|\theta^{\mu})$ **Critic:** $Q(s, a|\theta^Q)$

12.3 TD3

Improvements to DDPG: 1. **Clipped Double Q:** $\min(Q_1, Q_2)$ 2. **Delayed Updates:** Update policy less frequently 3. **Target Smoothing:** Add noise to target actions

12.4 SAC (Soft Actor-Critic)

Maximum entropy RL: $J(\pi) = \mathbb{E}_{\pi}[r + \alpha H(\pi(\cdot|s))]$ where $H(\pi) = -\sum_a \pi(a|s) \log \pi(a|s)$

13 Key Algorithms Comparison

Method	Model	Policy	Bias/Var
MC	Free	On/Off	Low/High
TD	Free	On/Off	High/Low
DP	Based	-	Low/Low

On-Policy: Same policy for learning and acting (SARSA, Policy Gradient) **Off-Policy:** Different policies (Q-Learning, Importance Sampling)

14 Key Convergence Results

Linear Semi-Gradient TD(0): Converges if A is positive definite **Tabular Q-Learning:** Converges w/ appropriate learning rates **Policy Gradient:** Converges to local optimum **Off-Policy + Function Approx:** Can diverge (Deadly Triad)